



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Gerber Mariano Pérez Gómez **Carné:** 1165825

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Es una interfaz que permite que dos sistemas se comuniquen a través de HTTP siguiendo los principios de la arquitectura REST. En dicha arquitectura, todo se modela como recursos que se identifican como URLs y se manipulan con HTTP estándar.

2. ¿Qué es un endpoint?

Es el punto de acceso concreto de una API donde se puede realizar una operación sobre un recurso. Combina una ruta con un método HTTP.

3. ¿Qué diferencia existe entre una URL y un endpoint?

La URL es la dirección completa y general de un recurso en la web. El endpoint es un concepto más específico: es la URL de la API junto con el método HTTP y la operación que ejecuta. La misma URL puede representar varios endpoints distintos.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

- HTTP GET: Consultar o leer datos, sin modificarlos.
- POST: crea un nuevo recurso.
- PUT: reemplaza un recurso completo.
- PATCH: modifica de forma parcial un recurso
- DELETE: elimina un recurso.

5. ¿Qué es JSON y por qué es común en las APIs?

JSON (JavaScript Object Notation) es un formato de texto ligero para representar datos con pares clave-valor, objetos y listas, es común porque es fácil de leer para las personas, liviano en comparación con XML, independiente del lenguaje y se integra de forma natural con JavaScript y con la web.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Es un número de tres dígitos que el servidor incluye en la respuesta para indicar el resultado de la petición.

- 200 OK: la petición se procesó de forma correcta.
- 404 NOT FOUND: el recurso solicitado no existe.
- 500 INTERNAL SERVER ERROR: ocurrió un fallo inesperado en el servidor.



7. ¿Qué información se puede encontrar en la respuesta de una API?

- Código de estado: indica si la operación fue exitosa o falló.
- Encabezados: metadatos como Content-Type, fecha, tamaño o información de caché.
- Cuerpo: los datos solicitados, normalmente en JSON, o un mensaje de error con su descripción.
- Tiempo de respuesta y tamaño: información útil para evaluar el rendimiento (la muestran herramientas como Postman).

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

- Permite probar los endpoints sin escribir código cliente.
- Ayuda a entender la estructura de las respuestas antes de integrarlas.
- Facilita detectar errores de la API de forma temprana.
- Permite guardar peticiones en colecciones y reutilizarlas.

9. ¿Qué es y para qué sirve Swagger UI?

Swagger UI es una herramienta que genera, a partir de una especificación OpenAPI, una documentación web interactiva de la API. Sirve para ver todos los endpoints disponibles, sus parámetros, cuerpos esperados y posibles respuestas, y además permite probar los endpoints directamente desde el navegador con el botón "Try it out".

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Postman es una plataforma para diseñar, probar, documentar y automatizar APIs. Su función principal es enviar peticiones HTTP a un servicio y mostrar la respuesta, lo que permite validar que la API funcione como se espera.



1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
/pet/{petId}
- Indique el método HTTP utilizado.
GET
- Indique qué parámetro necesita la solicitud.
Se requiere el parámetro petId. Es un parámetro de ruta, es decir, su valor se incluye directamente en la URL de la petición.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
200 OK
- Explique con sus palabras qué información contiene la respuesta.
Un JSON que representa el recurso solicitado. Este incluye el campo id (identificador único), category (objeto con id y name), name, photoUrls (arreglo de URLs de imágenes), tags (arreglo de etiquetas) y status, cuyos valores posibles son available, pending o sold.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

The screenshot shows the Swagger UI for the Petstore3 API. The endpoint `GET /pet/{petId}` is selected. The parameter `petId` is set to `10`. The response is a 200 OK status with a JSON body:

```
{
  "id": 10,
  "name": "Buffy",
  "photoUrls": [
    "https://petstore3.swagger.io/api/v3/imgs/pet10.jpg"
  ],
  "tags": [
    "cat"
  ],
  "status": "available"
}
```

The response headers are also visible:

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 62
content-type: application/json
date: Fri, 03 Oct 2025 18:28:16 GMT
server: Jetty(9.4.53.v20231003)
```



3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

The screenshot displays the Swagger UI interface in a web browser. The top navigation bar shows the URL `petstore3.swagger.io/#/pet/addPet`. The main content area is divided into two sections: 'Edit Value' and 'Responses'.

In the 'Edit Value' section, a JSON object is entered into the 'Body' field:

```
{
  "id": 1000,
  "name": "Rafael",
  "status": "available",
  "category": "dog",
  "tags": [
    "tag1", "tag2"
  ]
}
```

Below the JSON input, there is a 'Execute' button. The 'Responses' section shows the result of the POST request:

200 Successful operation

The response body is a JSON object:

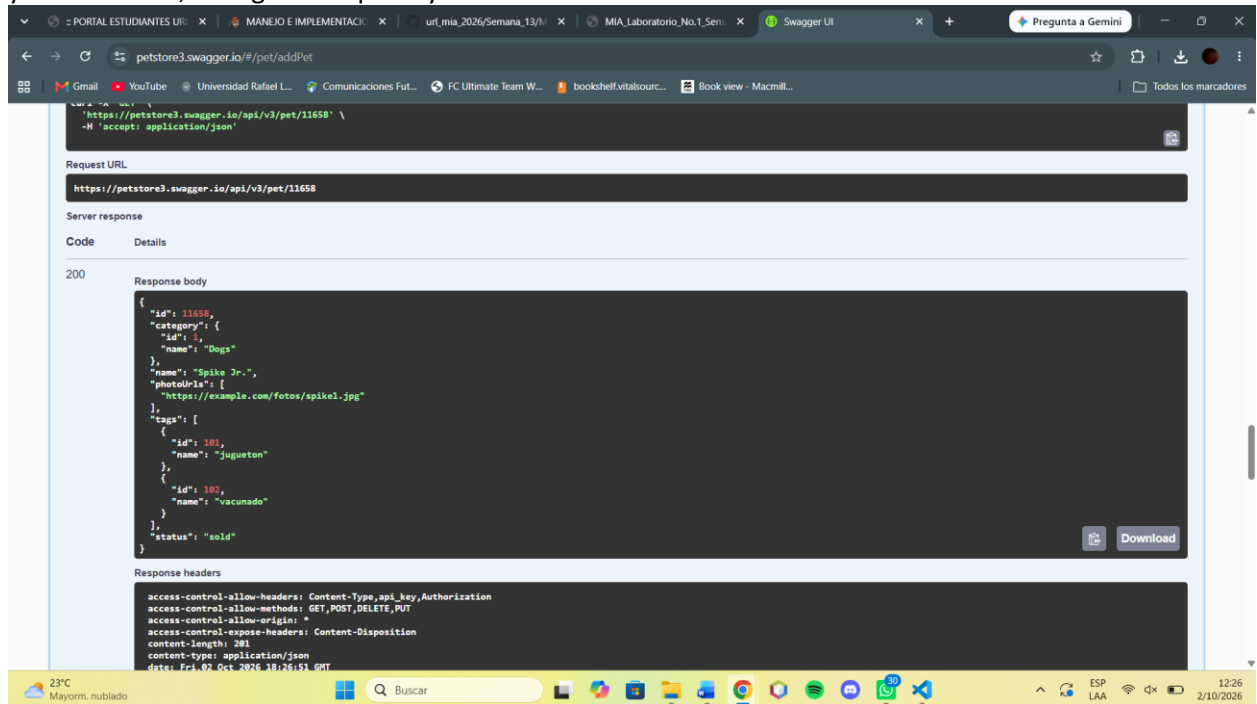
```
{
  "id": 1000,
  "name": "Rafael",
  "status": "available",
  "category": "dog",
  "tags": [
    "tag1", "tag2"
  ]
}
```

The bottom part of the screenshot shows the 'Responses' section for the GET /pet/{petId} endpoint, which returns a 400 Invalid input error.

```
{
  "id": 11658,
  "name": "Spike Jr.",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "https://example.com/fotos/spike1.jpg"
  ],
  "tags": [
    {
      "id": 101,
      "name": "jugueton"
    },
    {
      "id": 102,
      "name": "vacunado"
    }
  ],
  "status": "sold"
}
```



- ¿Qué código HTTP devolvió la API?
Devolvió 200 OK, junto con un cuerpo que repite los datos actualizados.
- ¿Cómo comprobó que el cambio realmente ocurrió?
Consulté nuevamente la mascota con GET /pet/11658. La respuesta fue un 200 OK con name "Spike Jr." y status "sold", en lugar de "Spike" y "available".



5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
El parámetro es status. Sus valores permitidos son available, pending y sold. En este caso se usó available.
- Explique dónde se envía ese parámetro.
Se envía como parámetro de consulta (Query Parameter), es decir, ¿al final de la URL después del signo ?, con el formato nombre=valor.
- Observe la estructura de la respuesta.
La respuesta es un arreglo JSON (una lista entre corchetes []) donde cada elemento es un objeto con los datos de una mascota: id, category (objeto con id y name), name, photoUrls (arreglo de URLs), tags (arreglo de objetos con id y name) y status. Todos los elementos devueltos tienen status igual a available.
- Indique qué sucede si cambia el valor del parámetro por pending.
La URL cambia a ...findByStatus?status=pending y la API devuelve otra lista, con la misma estructura, pero solo con las mascotas cuyo estado es pending



petstore3.swagger.io/#/pet/findPetsByStatus

Parameters

Name	Description
status *required	Status values that need to be considered for filter
string	available
(query)	

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  https://petstore3.swagger.io/api/v3/pet/findPetsByStatus?status=available \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/findPetsByStatus?status=available

Server response

Code Details

200

Response body

```
{
  "id": 1001,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Buddy",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "tag"
    }
  ],
  "status": "available"
}
```

22°C Mayom, nublado

Status pending

petstore3.swagger.io/#/pet/findPetsByStatus

Parameters

Name	Description
status *required	Status values that need to be considered for filter
string	pending
(query)	

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  https://petstore3.swagger.io/api/v3/pet/findPetsByStatus?status=pending \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/findPetsByStatus?status=pending

Server response

Code Details

200

Response body

```
{
  "id": 1001,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Buddy",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "tag"
    }
  ],
  "status": "pending"
},
{
  "id": 1002,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Doggie",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "tag"
    }
  ],
  "status": "pending"
}
```

22°C Mayom, nublado



6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

POST:

The screenshot shows the Postman interface with a new POST request configured. The URL is `https://petstore.swagger.io/v2/pet`. The body is set to raw JSON. The JSON body is:

```
1 {
2   "id": 11658,
3   "name": "Spike Jr.",
4   "category": {
5     "id": 1,
6     "name": "Dogs"
7   },
8   "photoUrls": [
9     "https://example.com/fotos/spike1.jpg"
10  ],
11  "tags": [
12    {
13      "id": 101,
14      "name": "jugueton"
15    }
16  ]
17 }
```

The response is shown as `200 OK` with a status of `84 ms` and `489 B`. The response body is also shown in JSON format:

```
1 {
2   "id": 11658,
3   "category": {
4     "id": 1,
5     "name": "Dogs"
6   },
7   "name": "Spike Jr.",
8   "photoUrls": [
9     "https://example.com/fotos/spike1.jpg"
10  ],
11  "tags": [
12    {
13      "id": 101,
14      "name": "jugueton"
15    }
16  ]
17 }
```



GET:

Postman interface showing a GET request to `https://petstore.swagger.io/v2/pet/11658`. The response is a JSON object:

```
{
  "id": 11658,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Spike Jr.",
  "photoUrls": [
    "https://example.com/fotos/spike1.jpg"
  ],
  "tags": [
    {
      "id": 101,
      "name": "jugueton"
    }
  ]
}
```

PUT:

Postman interface showing a PUT request to `https://petstore.swagger.io/v2/pet`. The response is a JSON object:

```
{
  "id": 11658,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Spike Jr.",
  "photoUrls": [
    "https://example.com/fotos/spike1.jpg"
  ],
  "tags": [
    {
      "id": 101,
      "name": "jugueton"
    },
    {
      "id": 102,
      "name": "vacunado"
    }
  ]
}
```



Postman interface showing a GET request to `https://petstore.swagger.io/v2/pet/findByStatus?status=available`. The response is a 200 OK with a JSON body containing pet information.

Key	Value	Description	Bulk Edit
status	available		

```
{
  "id": 922337201690017785,
  "category": {
    "id": 22,
    "name": "uselessly less frivolous"
  },
  "name": "American Foxhound",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,

```

DELETE - Eliminar mascota

Postman interface showing a DELETE request to `https://petstore.swagger.io/v2/pet/11658`. The response is a 200 OK with a JSON body containing error information.

```
{
  "code": 200,
  "type": "unknown",
  "message": "11658"
}
```

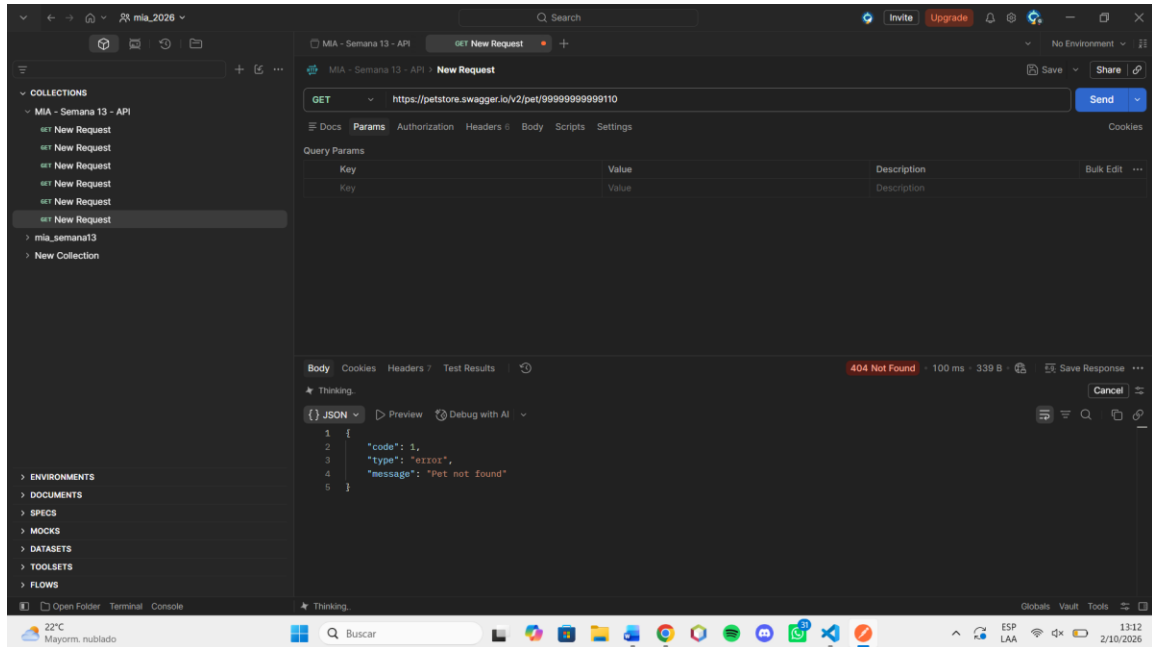


7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.

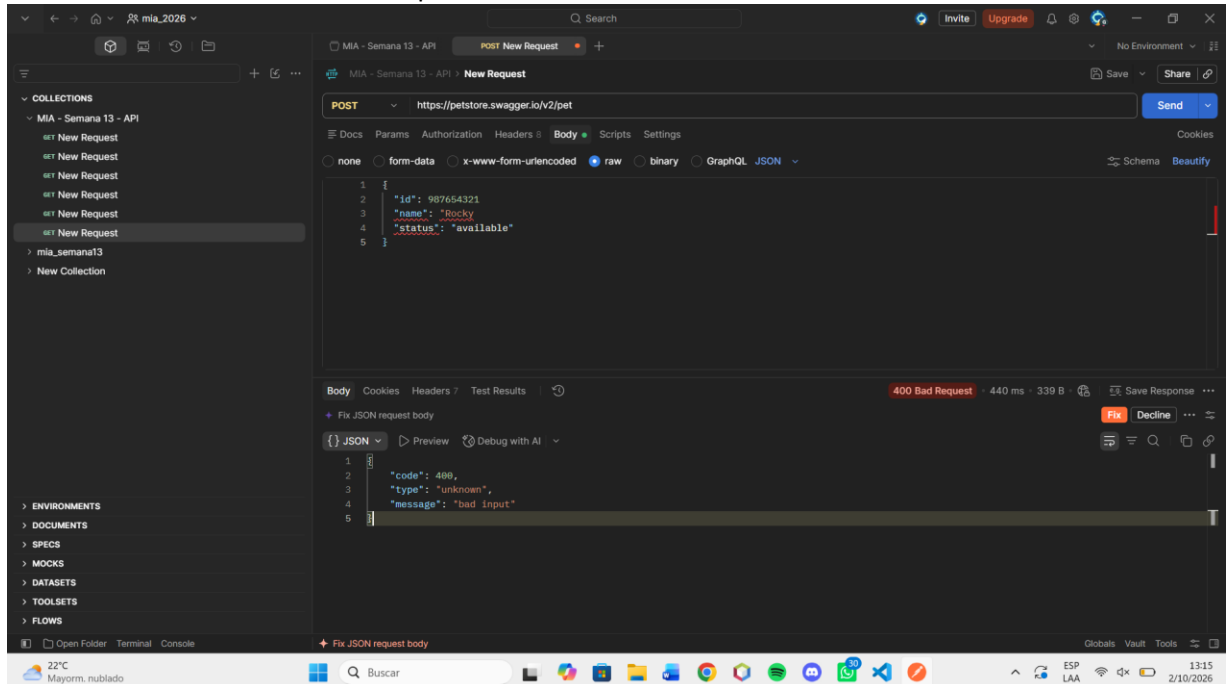


- Solicitud realizada.
<https://petstore.swagger.io/v2/pet/99999999999110>
- Código HTTP recibido.
404 Not Found
- Respuesta de la API.

```
{  \"code\": 1,  \"type\": \"error\",  \"message\": \"Pet not found\"}
```
- Qué esperaba que ocurriera.
Un error indicando que el recurso no existe.
- Qué ocurrió realmente.
La API respondió 404 con el mensaje "Pet not found".
- Explicación de la causa
No hay ninguna mascota registrada con ese ID, y 404 es el código estándar para un recurso inexistente.



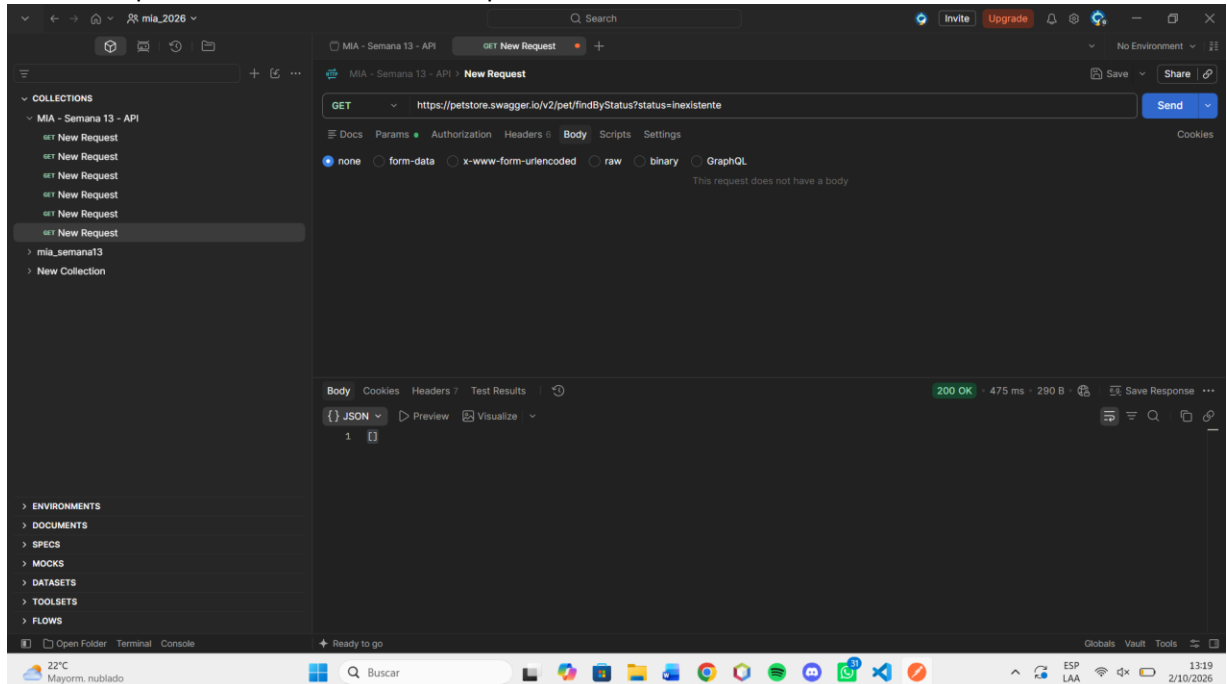
2. Enviar un JSON incorrecto o incompleto.



- Solicitud realizada.
POST <https://petstore.swagger.io/v2/pet>
Con el body roto (falta una coma y una comilla)
{ "id": 987654321 "name": "Rocky"status": "available" }
- Código HTTP recibido.
400 Bad Request
- Respuesta de la API.
{
 "code": 400,
 "type": "unknown",
 "message": "bad input"
}
- Qué esperaba que ocurriera.
Que la API rechazara la solicitud sin crear nada.
- Qué ocurrió realmente.
Devolvió error en lugar de 200. Un GET a ese ID confirma que no se creó.
- Explicación de la causa
El servidor no puede interpretar el body como JSON válido y lo rechaza antes de procesarlo.



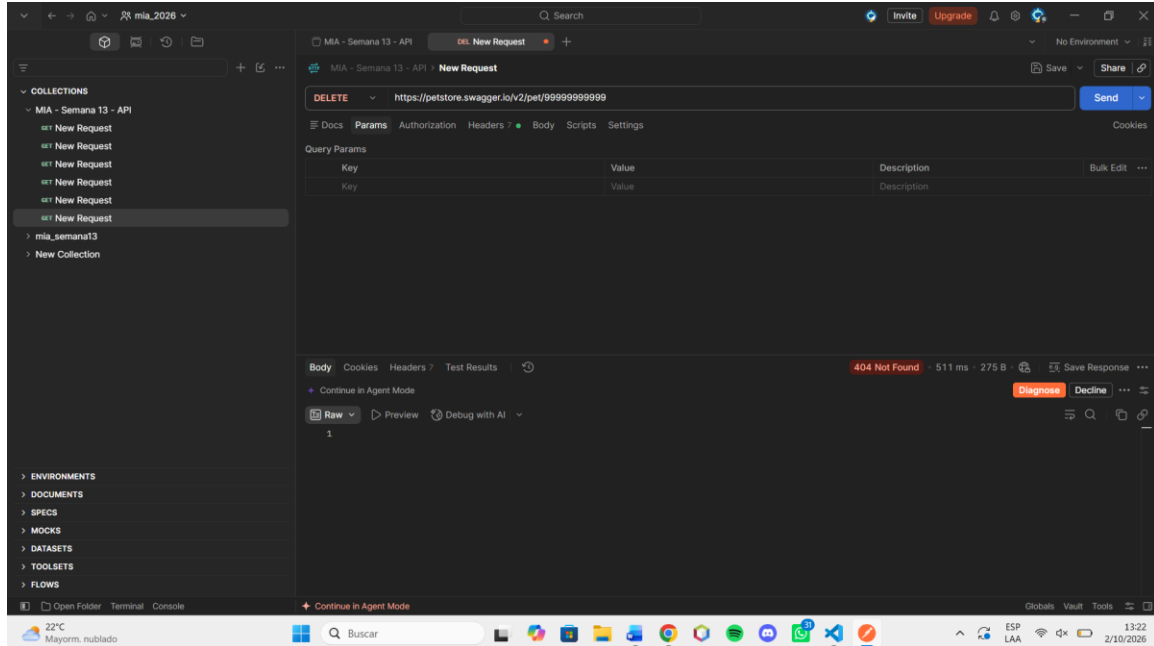
3. Utilizar un parámetro con un valor no esperado.



- Solicitud realizada.
GET <https://petstore.swagger.io/v2/pet/findByStatus?status=inexistente>
- Código HTTP recibido.
200 OK
- Respuesta de la API.
[]
- Qué esperaba que ocurriera.
Un 400 por valor inválido, ya que la documentación solo admite available, pending y sold.
- Qué ocurrió realmente.
No existió error, solo una lista vacía.
- Explicación de la causa
El servidor no valida el valor contra la lista permitida; filtra, no encuentra coincidencias y devuelve un arreglo vacío.



4. Intentar eliminar un recurso que ya no existe.



- Solicitud realizada.
<https://petstore.swagger.io/v2/pet/999999999999>
con header api_key: special-key
- Código HTTP recibido.
404 Not Found
- Respuesta de la API.
Cuerpo vacío
- Qué esperaba que ocurriera.
Un error, ya que no hay nada que borrar.
- Qué ocurrió realmente.
404, a diferencia del DELETE de una mascota existente, que devuelve 200.
- Explicación de la causa
El ID no corresponde a ningún recurso



8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

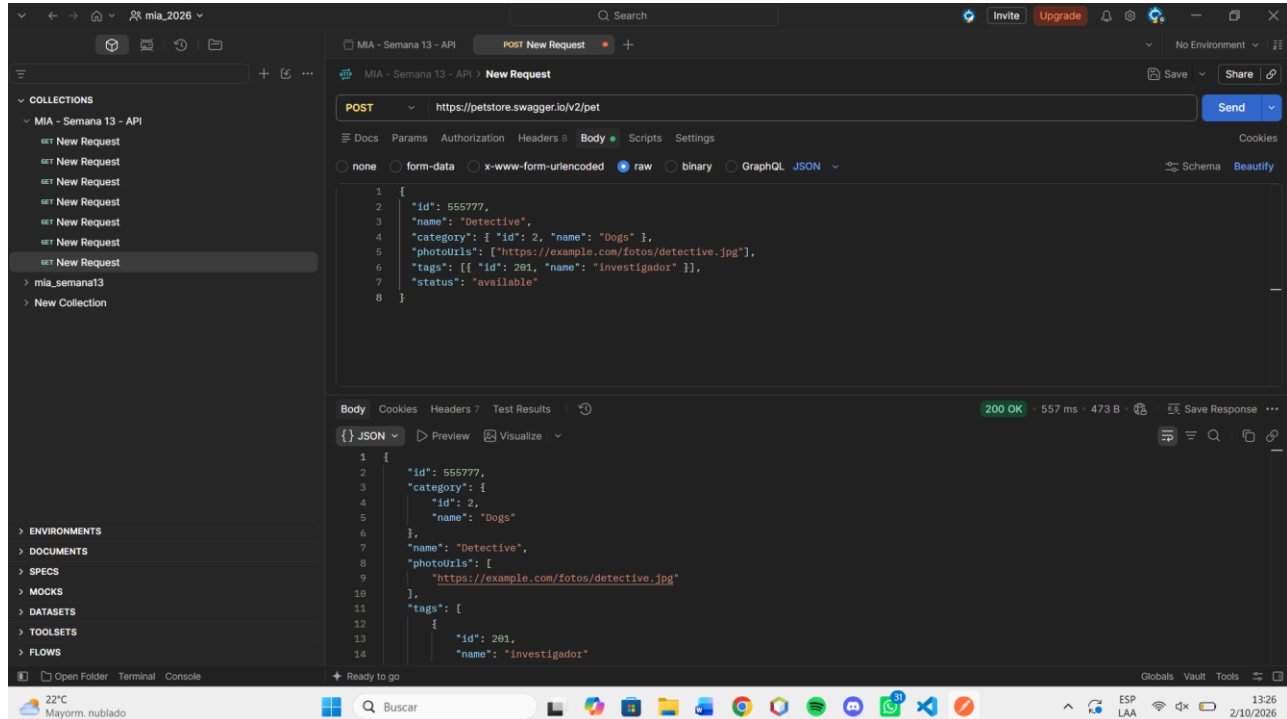
“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

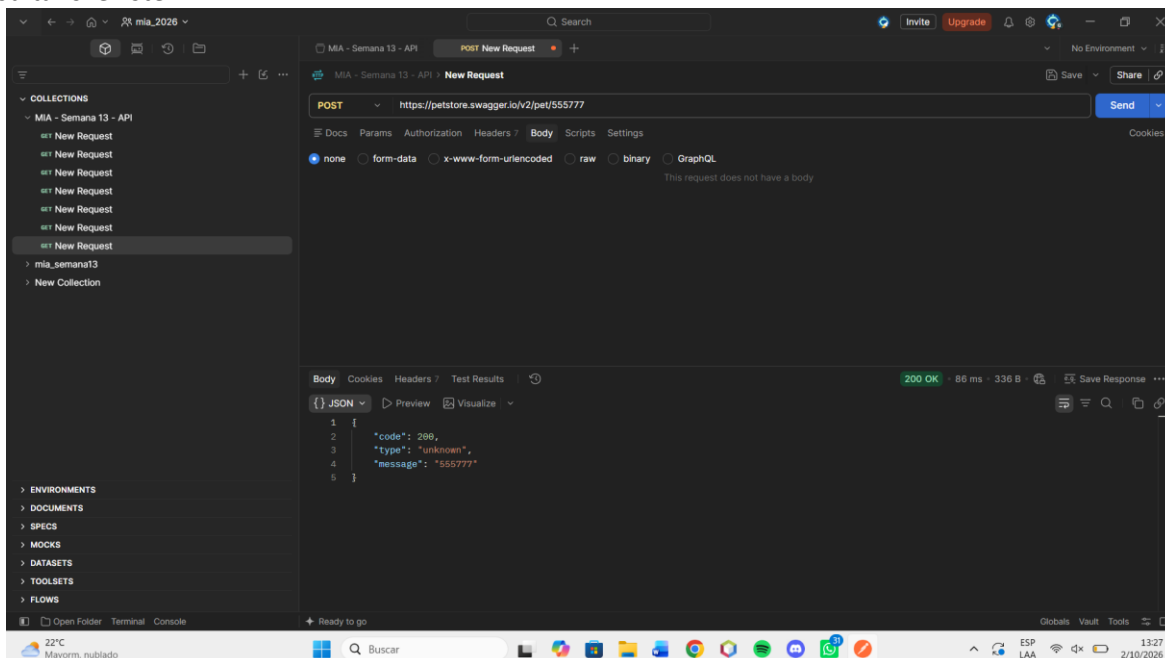
- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Creación de la mascota



Consultar si existe





The screenshot shows the Postman interface with a PUT request to `https://petstore.swagger.io/v2/pet`. The request body is a JSON object representing a pet. The response is a 200 OK status with a JSON body.

```
PUT https://petstore.swagger.io/v2/pet
```

```
{
  "id": 555777,
  "name": "Detective",
  "category": { "id": 2, "name": "Dogs" },
  "photoUrls": [ "https://example.com/fotos/detective.jpg" ],
  "tags": [ { "id": 201, "name": "Investigador" } ],
  "status": "sold"
}
```

```
{
  "id": 555777,
  "category": {
    "id": 2,
    "name": "Dogs"
  },
  "name": "Detective",
  "photoUrls": [
    "https://example.com/fotos/detective.jpg"
  ],
  "tags": [
    {
      "id": 201,
      "name": "Investigador"
    }
  ],
  "status": "sold"
}
```

Verificar el cambio

The screenshot shows the Postman interface with a GET request to `https://petstore.swagger.io/v2/pet/555777`. The response is a 200 OK status with a JSON body.

```
GET https://petstore.swagger.io/v2/pet/555777
```

```
{
  "name": "Dogs",
},
{
  "name": "Detective",
  "photoUrls": [
    "https://example.com/fotos/detective.jpg"
  ],
  "tags": [
    {
      "id": 201,
      "name": "Investigador"
    }
  ],
  "status": "sold"
}
```



Eliminar

The screenshot shows the Postman interface with a DELETE request to `https://petstore.swagger.io/v2/pet/555777`. The response is a 200 OK status with a JSON body: `{\"code\": 200, \"type\": \"unknown\", \"message\": \"555777\"}`. The left sidebar shows a collection named 'MIA - Semana 13 - API' with several 'New Request' items. The bottom status bar shows 'Ready to go'.

Confirmar que ya no existe

The screenshot shows the Postman interface with a GET request to `https://petstore.swagger.io/v2/pet/555777`. The response is a 404 Not Found status with a JSON body: `{\"code\": 1, \"type\": \"error\", \"message\": \"Pet not found\"}`. The left sidebar shows the same collection 'MIA - Semana 13 - API'. The bottom status bar shows 'Cre'.



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.

Swagger UI interface showing the Swagger Petstore - OpenAPI 3.0 API definition. The interface includes a search bar, a list of endpoints, and a detailed view of the 'pet' endpoint. The 'pet' endpoint is a GET request that returns a single pet. The response is a JSON object with fields: id, name, and category.

Swagger UI interface showing the 'Try it out' section for the 'pet' endpoint. It displays the request parameters, the response status (200), and the response body (JSON object).

Parameters:

Name	Description
petid * required	ID of pet to return

Responses:

Code	Description	Links
200	successful operation	No links

Media type: application/json

Example Value:

```
{  "id": 10,  "name": "doggie",  "category": {}}
```